

category: react-web

tags: [promise-chain, race-condition, generation-token, speechSynthesis, vanilla-js]

severity: medium

created: 2026-07-31

updated: 2026-07-31

project-origin: EnglishTool (LangTool)

Chuỗi `.then()` cũ vẫn chạy tiếp sau khi state đã đổi → tự cắt ngang việc mới bắt đầu (cần generation token)

Tình huống gặp phải

| Đang làm gì? Tính năng gì? Môi trường nào?

Focus Mode (`focus.js`) đọc nối tiếp nhiều đoạn TTS mỗi lần đổi từ: `speakCurrent()` build 1 chuỗi `Promise.then()` dài (từ → nghĩa → đánh vần (2 lần, có `setTimeout` 500ms giữa) → ví dụ). Chuỗi này **không được await hết** trước khi cho phép đổi từ tiếp theo — bộ đếm giờ mỗi từ (`setInterval` riêng) hết giờ đọc lập, gọi `show()` → gọi lại `speakCurrent(true)` cho từ MỚI trong khi chuỗi của từ CŨ có thể vẫn còn đang dở (đặc biệt khi tổng thời lượng đọc > thời gian mỗi từ được cấu hình, hoặc khi timer đọc lập định kỳ `repeatSec` cũng gọi `speakCurrent()`).

Triệu chứng / Lỗi

| Lỗi hiện ra như thế nào? Exception message? Behavior sai?

Không exception. User báo "lỗi khi hết thời gian chờ mà đang đọc dở" — âm thanh bị cắt ngang/lấn lộn giữa 2 từ: từ mới vừa bắt đầu đọc thì bị cắt ngang bởi 1 lệnh đọc "trễ" thuộc về từ trước đó vẫn đang nằm chờ trong `.then()/setTimeout` cũ.

Nguyên nhân gốc rễ (Root Cause)

`U.speak()` gọi `speechSynthesis.cancel()` **mỗi lần được gọi**, để dọn hàng đợi trước khi đọc utterance mới. Khi có 2 chuỗi `.then()` đọc lập cùng tồn tại (chuỗi của từ cũ chưa kết thúc + chuỗi của từ mới vừa bắt đầu), **chuỗi nào tới lượt gọi `U.speak()/U.speakSpelled()` sau cùng sẽ `cancel()` luôn audio của chuỗi kia** — bất kể chuỗi nào "mới hơn" về mặt UI. Không có cơ chế nào đánh dấu 1 chuỗi là "đã lỗi thời, đừng gọi TTS nữa" khi state (từ đang hiển thị) đã đổi.

Đây là dạng lỗi race-condition kinh điển: **fire-and-forget async chain không bị huỷ khi điều kiện tạo ra nó không còn đúng nữa.**

Giải pháp

Dùng **generation/epoch token**: mỗi lần bắt đầu 1 chuỗi mới, tăng 1 biến đếm dùng chung (`speechGen`) và chỉ tiếp tục các bước gọi TTS tiếp theo nếu token của chuỗi đó vẫn còn khớp với giá trị hiện tại — nếu không, coi như "đã lỗi thời", bỏ qua bước đó (không gọi thêm bất kỳ API side-effect nào):

```
let speechGen = 0;

function speakCurrent(full) {
  const w = current(); if (!w) return Promise.resolve();
  const gen = ++speechGen; // đánh dấu chuỗi này là "mới nhất"
  const alive = () => gen === speechGen; // còn là chuỗi mới nhất không?

  let p = U.speak(w.word, { rate: 1 }); // bước ĐẦU luôn hợp lệ – nó VỪA trở thành gen mới nhất
  if (c.speakSlow) p = p.then(() => alive() && U.speak(w.word, { rate: 0.6 }));
  if (full && c.spellWord) {
    p = p.then(() => alive() && U.speakSpelled(w.word))
      .then(() => alive() && new Promise(r => setTimeout(r, 500)))
      .then(() => alive() && U.speakSpelled(w.word));
  }
  return p;
}
```

Mọi nơi khác gọi lại `speakCurrent()` (đổi từ do hết giờ, đọc lặp định kỳ, bấm  đọc lại, next/prev thủ công) tự động tăng `speechGen` → tự vô hiệu hoá chuỗi cũ, KHÔNG cần biết/theo dõi chuỗi cũ đang ở bước nào.

Khi rời màn hình hẳn (cleanup/unmount), cũng tăng `speechGen` (+ `stopSpeak()`) để chuỗi cũ dở dang không "hồi sinh" phát tiếp sau khi user đã thoát.

Áp dụng lại (How to reuse)

- Bất kỳ đâu build 1 chuỗi `.then()` dài đại diện cho "công việc gắn với 1 state/item cụ thể" (đọc TTS cho 1 từ, fetch data cho 1 tab đang chọn, animation cho 1 card) mà **có thể bị super-sede bởi 1 lần gọi mới trước khi chuỗi cũ chạy xong** → PHẢI có generation token, đừng giả định chuỗi cũ sẽ "tự nhiên" không còn ảnh hưởng gì chỉ vì state đã đổi.
- Dấu hiệu cần token: hàm có `setTimeout`/nhiều `await` liên tiếp VÀ có thể được gọi lại nhiều lần trong lúc lần gọi trước còn dang dở (do timer độc lập, do user thao tác nhanh, do effect chạy lại).
- Đặt biến gen ở scope NGOÀI hàm (per-session/per-component), tăng nó ngay ĐẦU hàm trước khi làm bất kỳ side-effect nào, check `alive()` trước MỌI side-effect tiếp theo trong chuỗi (không chỉ check 1 lần ở đầu).

Chú ý / Cạm bẫy (Gotchas)

- ⚠ Bước ĐẦU TIÊN của chuỗi mới không cần (và không nên) check `alive()` — nó luôn hợp lệ vì chính nó vừa gán giá trị mới nhất cho gen; chỉ các bước SAU mới cần check.
- ⚠ `alive() && sideEffectCall()` trong `.then()` — nếu `alive()` false, biểu thức trả về `false` (không phải Promise) nhưng vẫn hợp lệ làm giá trị resolve của `.then()`, chuỗi tiếp tục "trôi qua" êm re mà không gọi thêm side-effect nào — không cần try/catch hay early-return phức tạp.
- ⚠ Đây KHÔNG thay thế việc `cancel()`/`abort()` công việc cũ đang thực sự chạy (VD utterance đang phát) — token chỉ chặn các bước TƯƠNG LAI của chuỗi cũ chưa chạy tới, còn bước hiện tại (nếu đang là network request/audio thật) vẫn cần cơ chế huỷ riêng (ở đây may mắn là `U.speak()` tự `cancel()` hàng đợi mỗi lần gọi mới nên không cần AbortController riêng).

Tham chiếu

- Liên quan: `[[web-speech-synthesis-chain-hangs-missing-voice]]` (cùng file `focus.js/utils.js`, phát hiện trong cùng phiên làm việc)
- Project liên quan: EnglishTool (`app/assets/js/focus.js`)